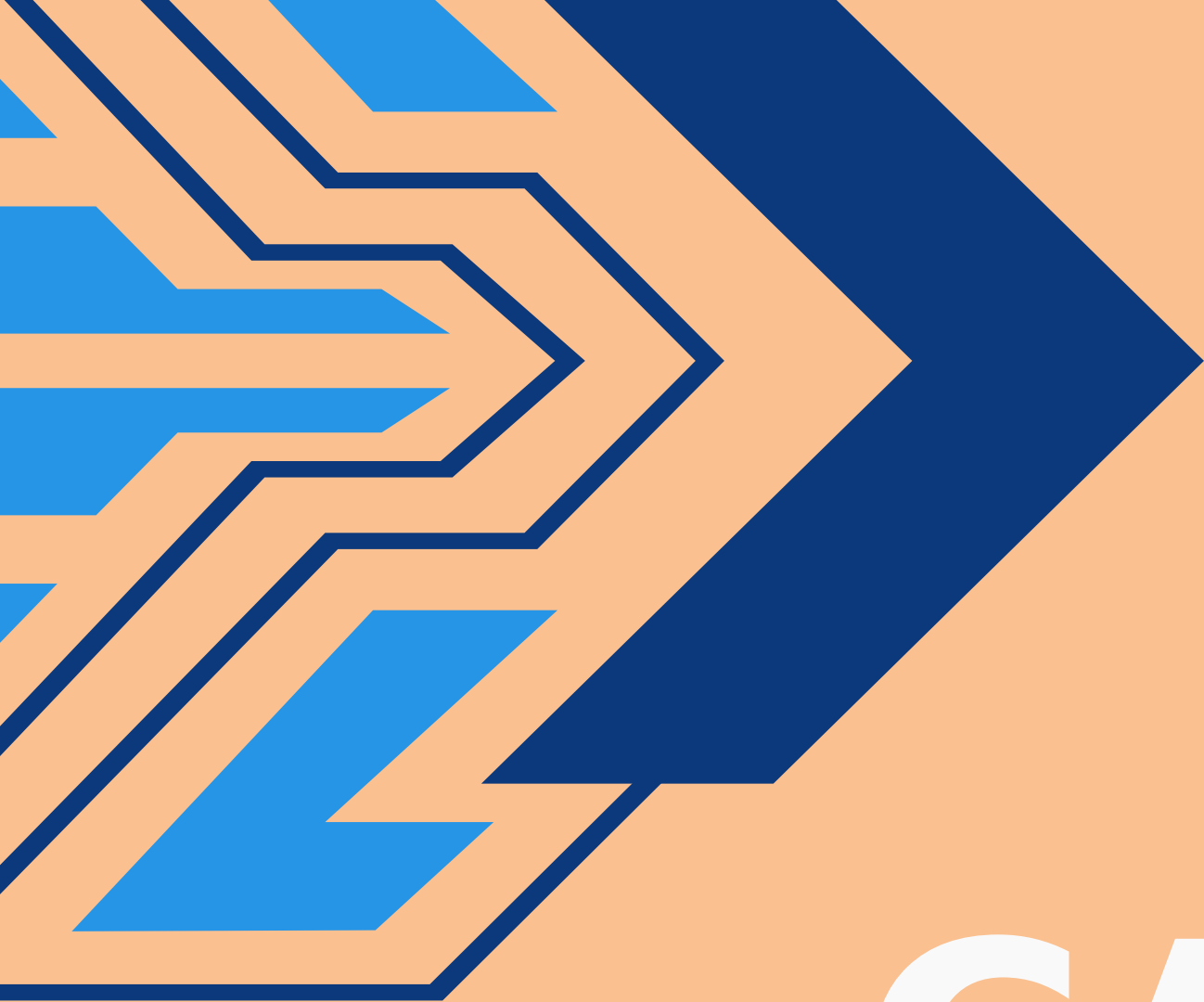




# SMART CALCULATOR PROJECT



# OBJECTIVES/ GOALS



1) LEARN ABOUT NEW SHORTCUTS AND LOGICAL OPERATORS

2) MASTER THE INTEGRATION OF PYTHON MATH MODULE

3) LEARN ON HOW TO TRACK DATA PERSISTENCE

4) KNOW ABOUT THE BUILT IN FUNCTIONS AND MODULE FUNCTIONS



# P R O B L E M S

1

- Handling invalid user input (wrong symbols, letters instead of numbers).

2

- Managing different operation types (single, multiple, currency).

3

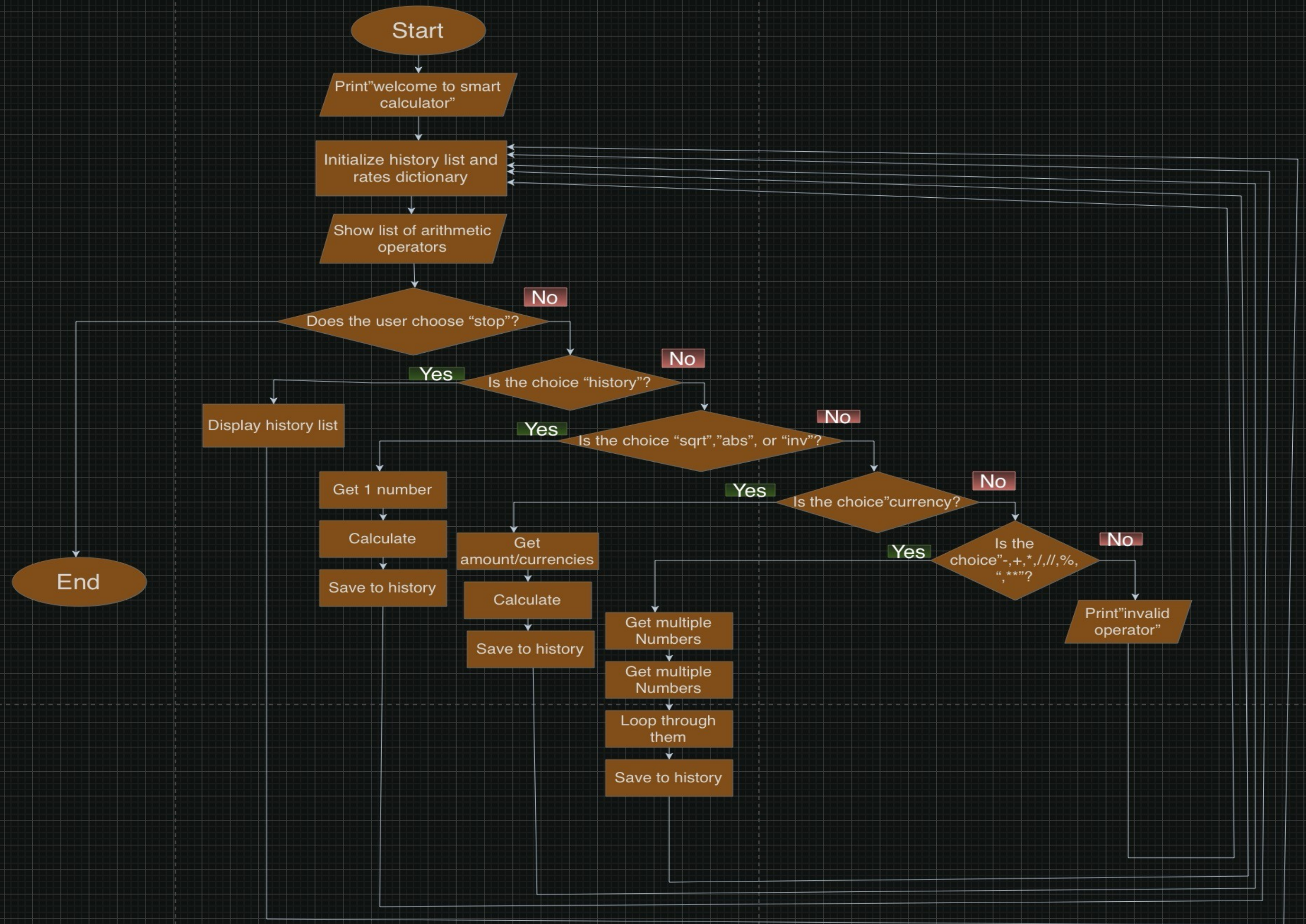
- Data formatting & cleanup and not confusing the program.

4

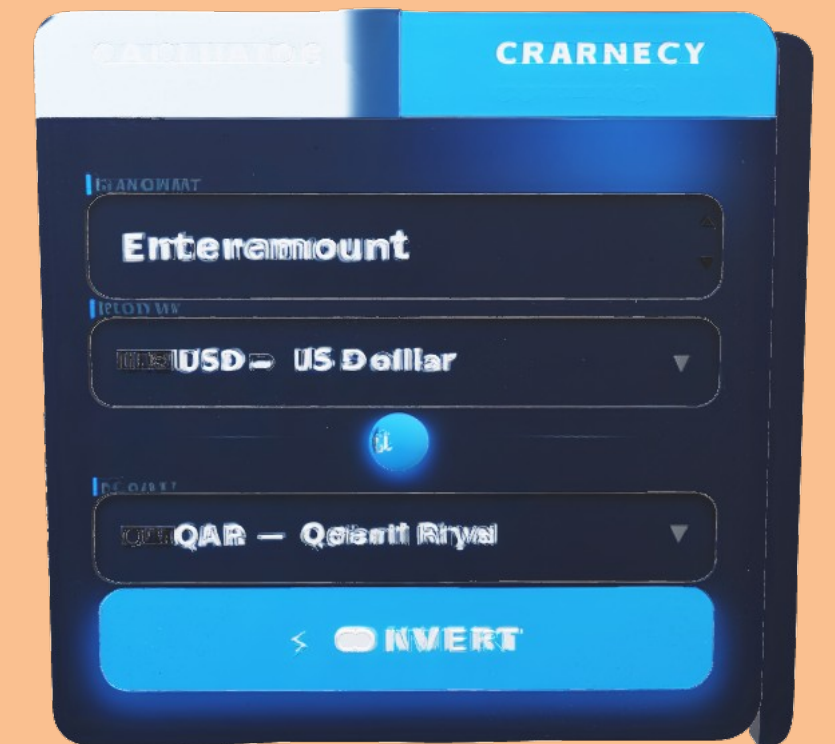
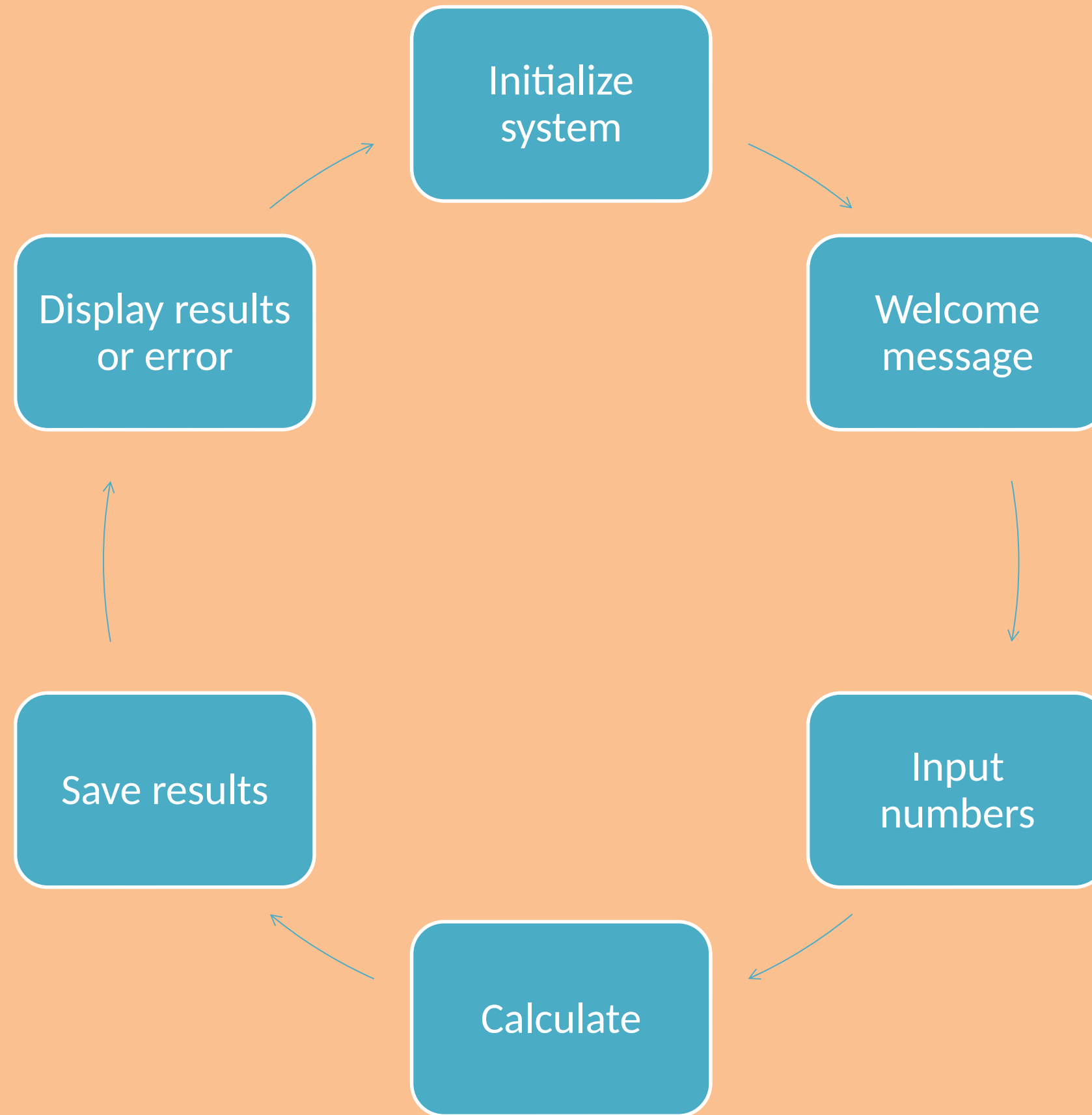
- Varying input requirements: some operations need one number(sort), while others need many.

# FLOWCHAR T





# The main idea of the flowchart:



# BUILT-IN FUNCTIONS

**PRINT():**

Displays text or  
variables to the  
output

**INPUT():**

Takes a string of  
data entered by  
the user

**FLOAT():**

Converts a string  
or integer into a  
decimal number

**ABS():**

Calculates the  
absolute  
(positive) value  
of a number

**LEN():**

Returns the  
number of  
items in a list

**ENUMERATE()**

:

Tracks the  
index and  
value of items  
in a list during  
a loop

**RANGE():**

Generates a  
sequence of  
numbers



# STRING & LIST METHODS, MODULES

## STRING & LIST METHODS:

- **lower() / .upper():** Converts text to all lowercase or uppercase.
- **strip():** Removes extra spaces from the beginning and end of a string.
- **split():** Breaks a string into a list based on spaces.
- **append():** Adds a new calculation result to your history list.

**Module used in the program:**

**math:** A built-in Python module used here specifically for the `math.sqrt()` function.

# SHORTCUTS & LOGICAL OPERATORS:

- **Augmented Assignment** (`+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=` ): Performs a math operation and updates the variable in one step.
- **List Comprehension**: The line `[float(num) for num in numbers]` is a shortcut for creating a new list by applying a function to every item in an existing one.
- **Slicing** (`[1:]`): A shortcut to select all items in a list starting from the second one.
- **try-except**: A shortcut for handling errors (like non-number inputs) without the program crashing.

# CODE & COMMENT

`import math` <— this module is built in python code as its a library in

`python`  
`def smart_calculator` <— def is for short define its a keyword that i can organize my coding only calling it once not writing the same steps over and over  
`# Print the welc`  
`print("Welcome t`  
`print("Available list of operators: +, -, *, /, // (integer division), % (modulus), ** (power)")`  
`print(" sqrt → Square Root (only first number used)")`  
`print(" abs → Absolute Value (only first number used)")`  
`print(" inv → Additive Inverse (only first number used)")`  
`print(" currency → Convert currencies")`  
`print(" history → Show all past results")`  
`print("Type 'stop' anytime to exit.\n")`

<—a print is a command that sends text or data to the screen

`history = []` <—this function creates an empty list that acts as program's memory, storing a list of past results so they can be viewed later.

`# Dictionary containing static exchange rates for currency conversion`

```
rates = {
    "USD": {"USD": 1, "EUR": 0.92, "GBP": 0.78, "QAR": 3.64},
    "EUR": {"USD": 1.09, "EUR": 1, "GBP": 0.85, "QAR": 3.95},
    "GBP": {"USD": 1.28, "EUR": 1.18, "GBP": 1, "QAR": 4.65},
    "QAR": {"USD": 0.27, "EUR": 0.25, "GBP": 0.21, "QAR": 1}
}
```

<—its a folder inside a folder, you pick a currency, then pick a target, and it gives you the exact conversation rate instantly; aka inception (key:value)

```
# Main loop to keep the calculator running
```

```
while True:
```

<—it keeps the program stays alive and running until the user says stop

```
# Get the operator choice from the user
```

```
operator_choice = input("Choose an operator (+, -, *, /, //, %, **, sqrt, abs, inv, currency, history): ").strip()
```

Lets the text from the user automatically in a lowercase |.

```
# Check if the user wa
```

< --

```
if operator_choice.lower() == "stop":
```

```
    print("Calculator shutting off. Goodbye!")
```

```
    break
```

<—a shortcut that cuts the loop and lets the program completely shutting off

```
# If choice is history, display the list of previous results
```

```
if operator_choice == "history":
```

```
    if history:
```

```
        print("Calculation History:")
```

```
        for index, result in enumerate(history, 1):
```

```
            print(f"{index}: {result}")
```

```
    else:
```

```
        print("No calculations yet.")
```

<—the "f"-string tells to python: hey look! Inside the curly brackets and swap the varubale names for the actual values

<—index, the current line number (1,2,3..)

<—the result, will print the actual math answer stored in that data

the continue shortcut will stop there and it will ignore any math or code on the terminal and it will keep looping

^  
|  
its a clean up tool; it takes the user's input and deletes accidental spaces front the start and end so the programs wont get confused



```
# Logic for currency conversion
```

```
if operator_choice == "currency":
```

```
    try:
```

```
        amount = float(input("Enter amount: "))
        from_currency = input("Enter from currency: ").strip().upper()
        to_currency = input("Enter to currency: ").strip().upper()

        # This piece of data is the Currency Converter.
        # It cleans up the user's text using .strip()
        # and .upper(), pulls the live exchange rate from
        # a dictionary, and uses :.2f to ensure the final
        # answer looks like actual money with only two
        # decimal places.
```

```
        # Check if currencies exist in the rates dictionary
```

```
        if from_currency not in rates or to_currency not in rates:
            print("Invalid currency choice!\n")
            continue
```

```
        result = amount * rates[from_currency][to_currency]
        print(f"Converted Amount: {result:.2f} {to_currency}\n")
        history.append(result)
```

```
    except ValueError:
```

```
        print("Invalid input. Please enter numbers only.\n")
```

```
    continue
```

```
        GBP, QAR): ").strip().upper()
```

```
        , QAR): ").strip().upper()
```

This section is the Currency Converter. It cleans up the user's text using .strip() and .upper(), pulls the live exchange rate from a dictionary, and uses :.2f to ensure the final answer looks like actual money with only two decimal places.

```
# Validate if the choice is a standard math operator
if operator_choice not in ["+", "-", "*", "/", "//", "%", "**"]:
    print("Invalid operator. Try again!\n")
    continue
```

The core idea here is Validation. We don't want the program to crash just because a user made a typo. By using the not in check, we filter out any invalid symbols right at the beginning. If the input is bad, we use continue to clear the mistake and give the user another chance without stopping the whole calculator.

This slide is about Multi-Input Handling. It takes a single line of text from the user, chops it into individual numbers, converts them from text to math-ready decimals, and ensures the user provided enough data (at least two numbers) to actually perform a calculation.

```
# Logic for multi-number calculations
try:
    numbers = input("Enter numbers separated by spaces: ").split()
    numbers = [float(num) for num in numbers]

    # Ensure the user provided at least two numbers for these operators
    if len(numbers) < 2:
        print("Please enter at least two numbers!\n")
        continue
```

```
result = numbers[0]
# Loop through numbers and apply the chosen operator sequentially
for num in numbers[1:]:
    if operator_choice == "+":
        result += num
    elif operator_choice == "-":
        result -= num
    elif operator_choice == "*":
        result *= num
    elif operator_choice == "/":
        if num == 0:
            print("Error: Division by zero!\n")
            break # Exit the for loop on division by
        result /= num
    elif operator_choice == "//":
        if num == 0:
            print("Error: Division by zero!\n")
            break
        result //= num
    elif operator_choice == "%":
        result %= num
```

- **Sequential Processing:** Automates math across a list of any size.
- **Optimized Logic:** Uses slicing to separate the "starting value" from the "math values."
- **Crash Prevention:** Includes a safety "Break" to handle invalid mathematical operations (Division by Zero) before they happen

```

    result -= num
elif operator_choice == "*":
    result *= num
elif operator_choice == "/":
    if num == 0:
        print("Error: Division by zero")
        break # Exit the for loop on error
    result /= num
elif operator_choice == "//":
    if num == 0:
        print("Error: Division by zero")
        break
    result //= num
elif operator_choice == "%":
    if num == 0:
        print("Error: Division by zero")
        break
    result %= num
elif operator_choice == "**":
    result **= num

```

- Diverse Toolset: Supports advanced math like Exponents (\*\*), Floor Division (//), and Remainders (%), moving beyond a basic "plus/minus" calculator.
- Total Safety Coverage: A dedicated "Zero Check" is placed in front of every division-related operation. This is Defensive Programming at its best.
- The "Emergency Exit" (break): If the user inputs a zero, the break command kills the calculation instantly. This prevents the program from attempting an "impossible" math problem and crashing.
- User Feedback: Instead of a silent error, it provides a clear "Division by zero!" message so the user knows exactly what went wrong.



```
    result = num  
else:  
    # This block runs only if the for loop completed without a 'break'  
    print(f"Result: {result}\n")  
    history.append(result)  
  
except ValueError:  
    print("Invalid input. Please enter numbers only.\n")
```

This is the "Success Gate." The else block only prints and saves the result to history if the math finished without errors. The except block is a safety shield that catches typos (like letters) to prevent the program from crashing.

```
# Run the calculator  
smart_calculator()
```

Here we just call the whole code when the program is about to start running

# RESULTS

```
Available list of operators: +, -, *, /, // (integer division), % (modulus), ** (power)
sqrt → Square Root (only first number used)
abs → Absolute Value (only first number used)
inv → Additive Inverse (only first number used)
currency → Convert currencies
history → Show all past results
Type 'stop' anytime to exit.

Choose an operator (+, -, *, /, //, %, **, sqrt, abs, inv, currency, history): inv
Enter a number: 90
Result: -90.0

Choose an operator (+, -, *, /, //, %, **, sqrt, abs, inv, currency, history): abs
Enter a number: 45
Result: 45.0

Choose an operator (+, -, *, /, //, %, **, sqrt, abs, inv, currency, history): sqrt
Enter a number: 125
Result: 11.180339887498949

Choose an operator (+, -, *, /, //, %, **, sqrt, abs, inv, currency, history): currency
Enter amount: 500
From currency (USD, EUR, GBP, QAR): QAR
To currency (USD, EUR, GBP, QAR): EUR
Converted Amount: 125.00 EUR
```

Converted amount: 125.00 EUR

Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): //  
Enter numbers separated by spaces: 7 7 8 7 5 4 4 7  
Result: 0.0

Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): \*  
Enter numbers separated by spaces: 0  
Please enter at least two numbers!

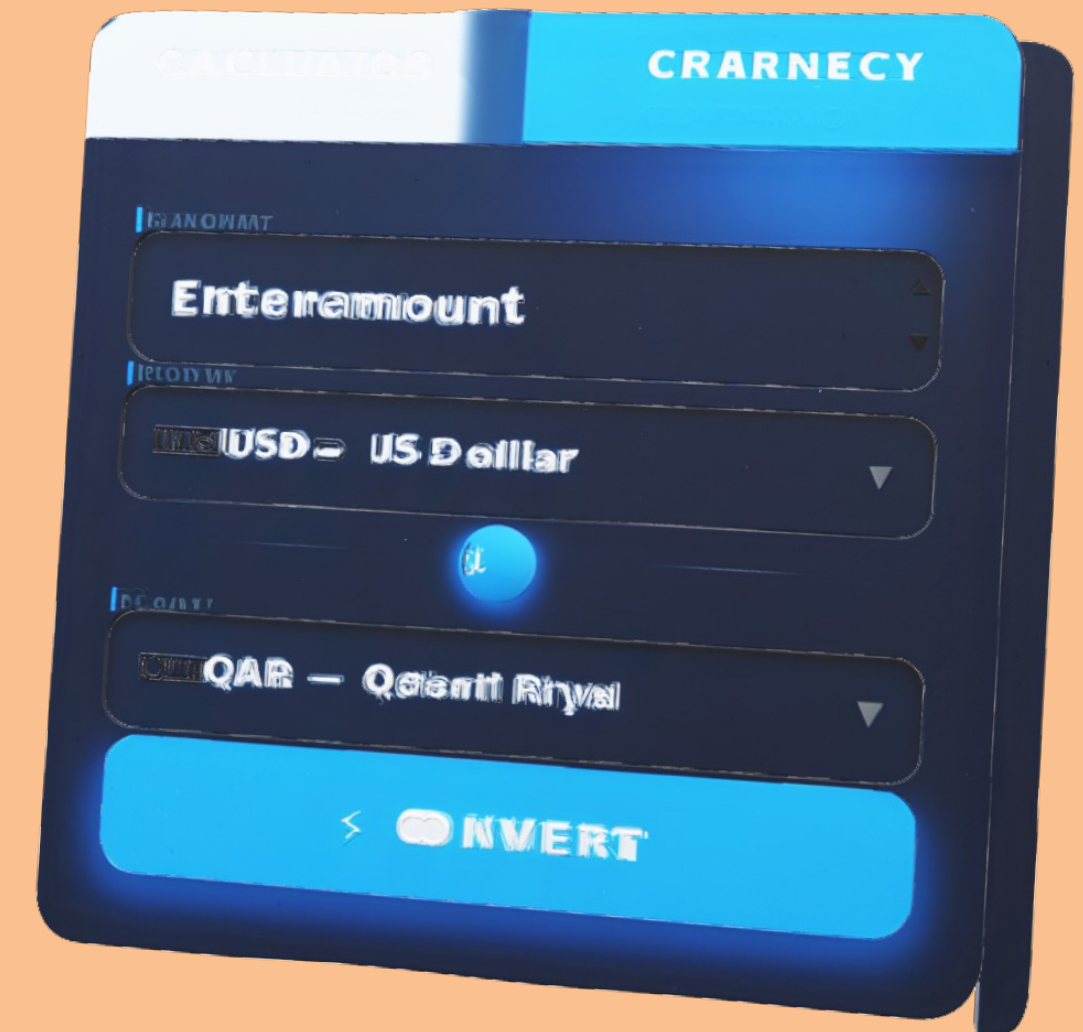
Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): \*  
Enter numbers separated by spaces: 0 4  
Result: 0.0

Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): /  
Enter numbers separated by spaces: 0 0  
Error: Division by zero!

Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): %  
Enter numbers separated by spaces: 0 4  
Result: 0.0

Choose an operator (+, -, \*, /, //, %, \*\*, sqrt, abs, inv, currency, history): stop  
Calculator shutting off. Goodbye!

# WEBSITE







THANK  
YOU

